Agentic Movie Recommender

1 Agentic Movie Recommendations

We've all been there – sitting down to watch a movie, and spending the night scrolling through endless options instead. Your job will be to build an **AI movie recommendation agent** that will solve this problem for your classmates.

For simplicity, you will be recommending movies from this database of about 800 of the most popular movies in recent years, taken from TMDB via their free API.

Deadline: your submissions are due 11:59 pm on Thursday, April 23.

2 Deliverables

2.1 Submission

Submit a **zip file** (under 10 MB) containing at minimum:

- `llm.py` — your implementation of `get_recommendation()`
- `requirements.txt` — one package per line
- `README.md` — explains your approach, evaluation strategy, and a brief guide to your code

You may include additional files (helper modules, data files, etc.) if your implementation needs them. Do not include `.env`, `.venv/`, or `__pycache__`.

Do NOT hard-code your API key. The grader will inject `OLLAMA_API_KEY` at run time. Your code must read it from the environment (`os.environ` or `os.getenv`). If you need additional environment variables, list them in a comment at the top of `llm.py`.

NOTICE To keep things fair, you may only use the `gemma4:31b-cloud` model when generating text. This is available for free via Ollama Cloud (see the starter code). If you find yourself exceeding their capacity during development (I don't think you will) you can have each team member use their own account for development, or pay for a premium subscription.

A starter implementation can be found here.

2.2 Function specification

Your submission must implement `get_recommendation()` in `llm.py`. The grader will call it with a user's preferences and watch history, and check the return value against the rules below.

Your goal is to convince the user that they will enjoy the movie you're recommending! That means selecting a good movie as well as writing a good description.

Input

```
def get_recommendation(preferences: str, history: list[str], history_ids: list[int] = []) -> dict:
```

Argument	Type	Description
`preferences`	`str`	Free-text description of what the user wants to watch
`history`	`list[str]`	Movie titles the user has already seen
`history_ids`	`list[int]`	TMDB IDs corresponding to `history`

Output

Return a `dict` of the form:

```
{
    "tmdb_id": 299536,
    "description": "infinity war raises the stakes of the entire marvel universe by pitting its grea
}
```

Where **description** tries to convince the user that this is something they would want to watch. Descriptions must be **500 characters or less** — longer descriptions will be truncated.

2.3 Competition Day

Before AI Agents Day 2 (see Syllabus for exact date), submit your zip file. The instructor will run all submissions against the same set of prompts.

Each of you will then serve as a judge for other teams' recommender systems. You will be presented with a recommendation from two randomly selected teams, and you will be asked to select a winner – whose recommendation do you think you're more likely to enjoy? (Or which did you enjoy more, if you've seen them.) In case both recommendations are the same, choose the more convincing description.

2.4 Automatic disqualifications

The following are prohibited and will automatically forfeit the round. **Read this carefully!**

1. Leaking the identity of your team or team members.
2. Taking longer than 20 seconds to return a response.
3. Recommending a movie that the user has already seen.
4. Returning a `tmdb_id` which is not in the candidate list.
5. Returning a value that is not a valid dict with `tmdb_id` and `description` keys.

2.5 Grading

You will be graded on the following criteria:

1. **Performance** (5pts): Based on your classmates' ratings of your recommendations, we will rank all of the agents submitted by your teams, and also a simple baseline agent. You will recieve points as follows:

- (2pts) Worse than baseline
- (3pts) Better than baseline
- (4pts) Better than baseline, and in the top 10 teams
- (5pts) Better than baseline, and in the top 3 teams

The top three teams will be called up in class to briefly describe their strategies and will recieve 3 points of extra credit.

2. **Evaluation** (5pts): When you're developing your agent, it will be critical for you figure out how to evaluate its outputs, and use this evaluation to optimize your agent. Your README.md must describe your evaluation strategy.

- (1-3 pts) No evaluation or poorly motivated evaluation
- (4pts) Well-motivated evaluation (e.g., LLM-as-a-judge)
- (5pts) Exceptionally creative evaluation process

3. **Creativity** (5pts): Instructors will evaluate the originality and sophistication of your approach — e.g., novel retrieval strategies, creative prompt engineering, automatic prompt optimization, interesting use of external data, or an unusually compelling description style. *The bare minimum (worth 3pts) is to try some prompts and add at least one tool.*

4. **Peer Review**: You will also be asked to complete review of your peers' effort and contributions. Failing to complete this review, or recieving consistently poor ratings from your peers, will result in your grade being scaled down.

3 Tips and tricks

- You can try any of the tricks that we've discussed in class, for example:
  - Hand-tuned prompt engineering: e.g., few-shot learning, chain-of-thought
  - Automatically-tuned prompt engineering using DSPy and GEPA
  - Retrieval augmented generation
- You may use external data sources, for example, calling the TMDB API.
- I expect you to make heavy use of Agentic AI to generate your recommendations. Try using Claude Code or OpenAI Codex – often you will simply be able to tell it your ideas in English, and it will execute it in code. You should also use at as an advisor regarding what to build, and how to do it in the simplest way possible.
- I encourage you to start with a simple hardcoded workflow, similar to the one already in the repo, and either tuning it, adding tools, or adding additional steps. If you happen to need more complexity, LangChain and DSPy are industry standard frameworks for building agents.